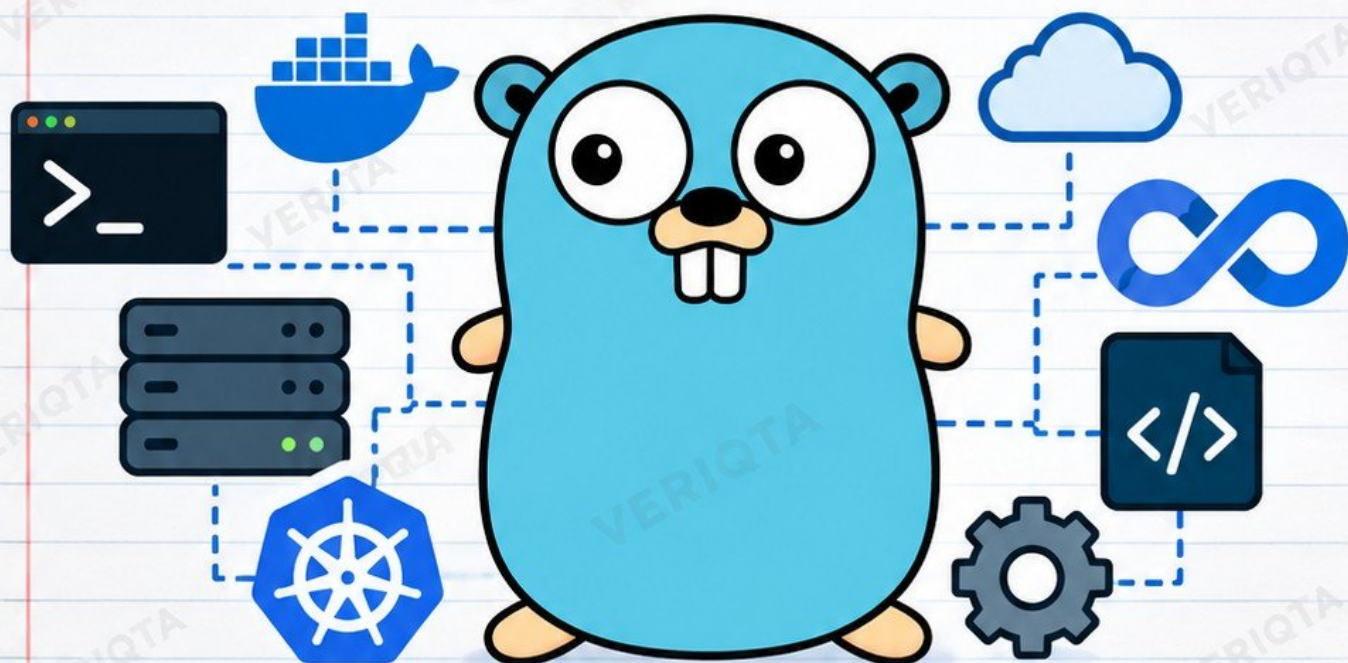


Go

for
**DevOps
Engineers**



**PRACTICAL
EXAMPLES**



**REAL DEVOPS
USE CASES**



**HANDS-ON
PROJECTS**



**PRODUCTION
FOCUS**

Why DevOps Engineers Learn Go

1. What is Go?

- Go is a modern, open-source programming language developed by Google.
- It was created to build fast, reliable and scalable software.
- Go was designed to be simple, efficient and easy to maintain.
- It focuses on real-world engineering needs such as performance, concurrency and large-scale systems.

2. Why was Go created?

- Existing languages were becoming complex for large systems.
- Build tools and infrastructure software were slow and hard to maintain.
- Google needed a language that was simple, fast and efficient.
- Go was designed to improve developer productivity and make it easier to build large, distributed systems.

3. Compiled vs Interpreted

Compiled (Go)

- Source code is compiled into a binary before running.
- Runs directly on the machine.
- Faster performance.
- Single executable file.
- Commonly used for infrastructure tools.

Interpreted (e.g. Python)

- Runs line by line using an interpreter.
- Needs the runtime environment.
- Slower performance.
- Requires dependencies to run.
- Commonly used for scripting and quick tasks.

4. Go in Cloud-Native Infrastructure

- Go is widely used to build cloud-native tools and infrastructure software.
- It handles high performance, concurrent tasks and large-scale environments.
- It has a simple syntax, strong standard library and excellent tooling.
- It is reliable, efficient and easy to deploy in modern infrastructure.

5. Single Binaries and Portability

- Go compiles to a single executable file.
- The binary includes everything needed to run the program.
- No external runtime is required.
- The same binary can run on different operating systems and architectures (Linux, Windows, macOS, containers).
- This makes deployment simple and reliable for DevOps and infrastructure tools.

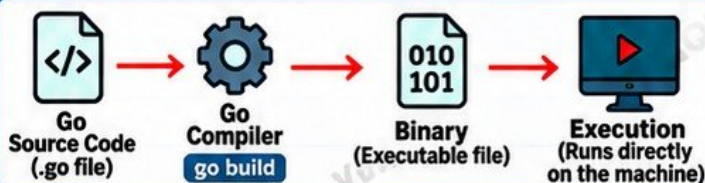
6. Where Go Appears in DevOps Work

- Automation scripts and CLI tools.
- Infrastructure management tools.
- Kubernetes operators and controllers.
- Monitoring and observability tools.
- Networking and service tools.
- Cloud provider tools and SDKs.
- CI/CD tools and custom tooling.

7. Common DevOps Tools Built with Go



8. Mental Model: How Go Works



Junior Engineer Tip

Learn Go to automate and build real tooling, not just to memorize syntax. Focus on solving DevOps and infrastructure problems.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Install Go and Build Your First Program

02 VERIQTA

1. Install Go

- 1 Go is available for Linux, macOS and Windows.
- 2 Download from the official site:
<https://go.dev/dl/>
- 3 Choose your operating system and follow the installation steps.
- 4 After installation, open a new terminal.

2. Verify the Installation

```
$ go version
go version go1.22.3 linux/amd64
```

- This command shows the installed Go version.
- If you see the version, Go is installed correctly.

3. Understanding .go Files

- Go source code is written in files with the **.go** extension.
- Each **.go** file contains Go code such as packages, imports and functions.
- You can create multiple **.go** files in a project.

4. Basic Program Structure

```
package main
import "fmt"
func main() {
    fmt.Println("Hello, DevOps!")
}
```

package main
Defines the package for an executable program.

import "fmt"
Imports the fmt package for input and output functions

func main() { }
The main function is the entry point of your program.

fmt.Println()
Prints text to the terminal.

5. Run the Program (Using Source Code)

```
$ go run main.go
Hello, DevOps!
```

- **go run** compiles and runs the code in one step.
- This is useful for quick testing and development.

6. Build a Compiled Binary

```
$ go build main.go
$ ./main
Hello, DevOps!
```

- **go build** compiles the code into a binary file.
- The binary can be executed directly.
- You can move the binary to other servers without needing Go installed.

7. Run vs Compiled Binary

	go run main.go	go build main.go
Command	Compiles and runs in one step	Creates a binary file
What it does	Runs the program immediately	Produces an executable binary (e.g. main)
Output	Development and quick testing	Production use, sharing and deployment
When to use		

8. Practice: DevOps Status Program

```
package main
import (
    "fmt"
    "time"
)
func main() {
    host, _ := os.Hostname()
    fmt.Println("DevOps Status Check")
    fmt.Println("Hostname:", host)
    fmt.Println("Time:", time.Now())
    fmt.Println("Status: All Good!")
}
```

Try This

- 1 Create a new file **status.go**
- 2 Add the code.
- 3 Run it using **go run status.go**
- 4 Build it using **go build status.go**
- 5 Execute the binary **./status**



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Variables, Constants, and Data Types

03



1. Variables in Go

- A variable stores a value in memory that can change.
- Two common ways to create variables:
 - `var` keyword (explicit declaration)
 - `:=` short syntax (declaration and assignment together)

```
var name string = "veriqta" // using var
var port int = 8080         // using var
env := "prod"              // using :=
retries := 3               // using :=
```

2. Constants with const

- A constant stores a value that cannot change.
- Use `const` for values that stay the same (e.g. configuration, limits).

```
const appName string = "myapp"
const defaultPort int = 8080
const maxRetries = 5
```

3. Go Data Types (Common Ones)

Type	Description	Example
string	Text data	"hello"
int	Whole numbers	8080
float64	Decimal numbers	3.14
bool	True or false	true / false

5. Type Inference

- Go can automatically detect the type based on the value.
- Use `:=` for short and clean code.

```
name := "api-server" // inferred as string
port := 9000         // inferred as int
healthy := true      // inferred as bool
latency := 2.5       // inferred as float64
```

7. Basic Type Conversion

- Convert between types when needed.

```
var portStr string = "8080"
portNum, _ := strconv.Atoi(portStr)
fmt.Println(portNum) // 8080

num := 10
str := strconv.Itoa(num)
fmt.Println(str)    // "10"
```

9. Common Mistake

- ❌ Confusing assignment with declaration.

- ❌ Wrong (redeclaring in same scope)

```
var port int = 8080
var port int = 9090 // error
```

- ✅ Correct (use assignment)

```
var port int = 8080
port = 9090 // update value
```

4. Examples with Different Types

```
var hostname string = "web-01" // server name
var port int = 8080           // service port
var cpuUsage float64 = 23.5    // CPU usage %
var isHealthy bool = true     // status check
```

6. Zero Values

- Uninitialized variables have a default value called the zero value.

Type	Zero Value
string	"" (empty string)
int	0
float64	0.0
bool	false

8. DevOps Examples

- Hostname (string): "web-01"
- Port (int): 8080
- Environment (string): "prod" / "dev"
- Retry count (int): 3
- Service status (bool): true / false
- Response time (float64): 0.256



10. Key Takeaways

- ✓ Use `var` or `:=` to create variables.
- ✓ Use `const` for values that don't change.
- ✓ Understand common data types.
- ✓ Go can infer types automatically.
- ✓ Zero values are the default for uninitialized variables.
- ✓ Convert types when needed.
- ✓ Use meaningful variable names in DevOps work.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Operators, Conditions, and Decisions

04



1. Comparison Operators

Operator	Meaning	Example	Result
<code>==</code>	Equal to	<code>5 == 5</code>	<code>true</code>
<code>!=</code>	Not equal to	<code>5 != 3</code>	<code>true</code>
<code>></code>	Greater than	<code>5 > 3</code>	<code>true</code>
<code><</code>	Less than	<code>3 < 5</code>	<code>true</code>
<code>>=</code>	Greater or equal	<code>5 >= 5</code>	<code>true</code>
<code><=</code>	Less or equal	<code>3 <= 5</code>	<code>true</code>

2. Logical Operators

Operator	Meaning	Example	Result
<code>&&</code>	AND (both true)	<code>true && true</code>	<code>true</code>
<code> </code>	OR (one true)	<code>true false</code>	<code>true</code>
<code>!</code>	NOT (reverse)	<code>!true</code>	<code>false</code>

- Use logical operators to combine multiple conditions.
- They are commonly used in DevOps automation (e.g. check multiple system states).

3. if, else if, and else

```
status := "down"

if status == "up" {
    fmt.Println("Service is up")
} else if status == "degraded" {
    fmt.Println("Service is degraded")
} else {
    fmt.Println("Service is down")
}
```

4. switch Statement

```
env := "prod"
switch env {
case "dev":
    fmt.Println("Development environment")
case "staging":
    fmt.Println("Staging environment")
case "prod":
    fmt.Println("Production environment")
default:
    fmt.Println("Unknown environment")
}
```

5. Combining Conditions

```
cpu := 75
memory := 80

if cpu > 70 && memory > 70 {
    fmt.Println("System under high load")
} else {
    fmt.Println("System is healthy")
}
```

6. Practical Example: Service Health Check

```
statusCode := 200
responseTime := 350

if statusCode == 200 && responseTime < 500 {
    fmt.Println("Service is healthy")
} else if statusCode >= 500 {
    fmt.Println("Service is down")
} else {
    fmt.Println("Service is slow or degraded")
}
```

7. Production Connection

- Automation often makes decisions based on system state, such as CPU usage, memory, status codes, response time, or disk space.
- Conditions and operators help your Go programs react to real-world infrastructure data.
- This is commonly used in monitoring tools, deployment scripts, health checks and CI/CD automation.



8. Junior Engineer Tip



Use conditions to make your programs behave correctly in different situations. Always think about what should happen when things go wrong.

9. Common Mistake



Confusing assignment with comparison.

```
if status = "up" { // wrong (this assigns a value)
    // ...
}
Use == for comparison:
if status == "up" { // correct
    // ...
}
```



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Loops and Repetitive Automation

05 VERIQTA

1. Go's for Loop

- Go uses only the for loop.
- It can behave like a traditional for loop, a while loop or an infinite loop depending on the condition.

```
for i := 1; i <= 5; i++ {  
    fmt.Println(i)  
}
```

2. Standard Loop (Counter Based)

- Used when you know how many times to run.
- Common in automation tasks like checking a list of servers.

```
for i := 0; i < 3; i++ {  
    fmt.Println("Checking server", i)  
}
```

3. Condition-Based Loop

- Runs while a condition is true.
- Often used when waiting for a system state.

```
count := 0  
for count < 5 {  
    fmt.Println("Retry:", count)  
    count++  
}
```

4. range Loop (Iterate Over Collections)

- Used to loop through slices, arrays, maps and other collections.
- Common for iterating through a list of services or servers.

```
services := []string{"web", "api", "db"}  
for i, svc := range services {  
    fmt.Println(i, "-", svc)  
}
```

5. break Statement

- Stops the loop immediately.
- Useful when a condition is met.

```
for i := 1; i <= 10; i++ {  
    if i == 5 {  
        fmt.Println("Stopping at", i)  
        break  
    }  
    fmt.Println(i)  
}
```

6. continue Statement

- Skips the current iteration and moves to the next one.
- Useful when you want to ignore certain items.

```
for i := 1; i <= 5; i++ {  
    if i == 3 {  
        continue // skip this iteration  
    }  
    fmt.Println("Value:", i)  
}
```

7. Practical Example: Check Multiple Services

- Iterate through a list of service names and print a simple status message.

```
services := []string{"nginx", "docker", "kubelet"}  
for _, svc := range services {  
    fmt.Println("Checking", svc, "...")  
    // here you can add real health check logic  
}
```

8. Common Beginner Mistake

- ✗ Accidentally creating an infinite loop.

```
for {  
    fmt.Println("This will run forever")  
}
```

- This loop has no condition and will never stop.
- Always use a condition or a **break** statement.

9. DevOps Use Cases

- Checking multiple servers or services
- Retrying an operation until it succeeds
- Processing a list of files or configurations
- Monitoring system state in a loop

10. Key Takeaways

- ✓ Use the **for** loop for all looping needs.
- ✓ Use **range** for collections.
- ✓ Use **break** to stop early.
- ✓ Use **continue** to skip an iteration.
- ✓ Be careful to avoid infinite loops.
- ✓ Loops are essential for automation in DevOps work.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Arrays, Slices, and Maps

06



1. Why Collections Exist

- Collections let you store multiple values of the same or different types.
- Useful for working with lists of servers, ports, environments, labels and configuration data.
- Make your code cleaner, easier to manage and more powerful.

2. Arrays vs Slices

Array (Fixed Size)

- Has a fixed length.
- Size is part of the type.
- Not flexible.

```
var ports [3]int = [3]int{
    80, 443, 8080
}
fmt.Println(ports)
```

Slice (Dynamic Size)

- Flexible size.
- Built on top of an array.
- Most commonly used in real projects.

```
ports := []int{80, 443}
ports = append(ports, 8080)
fmt.Println(ports)
```

3. Creating and Modifying Slices

- Create a slice using `[]type{...}`
- Add items using `append()`.
- Get the number of items using `len()`.

```
services := []string{"nginx", "api"}
services = append(services, "db")
fmt.Println(services) // [nginx api db]
fmt.Println(len(services)) // 3
```

4. append() and len()

Using append()

- Adds one or more items to a slice.
- Returns a new slice (with the added items).

```
nums := []int{1, 2}
nums = append(nums, 3, 4)
fmt.Println(nums)
// [1 2 3 4]
```

Using len()

- Returns the number of items in a slice, array or map.

```
items := []string{"a", "b", "c"}
fmt.Println(len(items))
// 3
```

5. Maps and Key-Value Data

- A map stores key-value pairs.
- Keys must be unique.
- Common for configuration, labels, environment variables and service details.

```
servers := map[string]string{
    "web-01": "nginx",
    "api-01": "api",
    "db-01": "postgres",
}
fmt.Println(servers)
```

6. Checking if a Map Key Exists

- Use the comma ok idiom.
- It tells you if a key exists in the map.

```
servers := map[string]string{
    "web-01": "nginx",
    "api-01": "api",
}
value, ok := servers["api-01"]
if ok {
    fmt.Println("Found:", value) // Found: api
} else {
    fmt.Println("Key not found")
}
```

7. DevOps Example: Services, Ports, Environments, Labels

- Use slices and maps to store real infrastructure data.

```
services := []string{"nginx", "api", "worker"}
ports := map[string]int{
    "nginx": 80,
    "api": 8080,
    "worker": 9000,
}
env := map[string]string{
    "nginx": "prod",
    "api": "staging",
    "worker": "prod",
}
fmt.Println(services)
fmt.Println(ports)
fmt.Println(env)
```

8. Practice: Service Inventory

- Create a small program to store and inspect services.

```
services := map[string]map[string]interface{}{
    "web-01": {"port": 80, "env": "prod", "status": "up"},
    "api-01": {"port": 8080, "env": "staging", "status": "down"},
    "db-01": {"port": 5432, "env": "prod", "status": "up"},
}
for name, details := range services {
    fmt.Println("Service:", name)
    fmt.Println("  Port:", details["port"])
    fmt.Println("  Environment:", details["env"])
}
```

9. Junior Engineer Tip



Use slices for lists of items and maps for key-value data. These are commonly used in DevOps for managing servers, ports, environments, labels and configuration values.

10. Common Mistake



Confusing an array with a slice.

- Arrays have a fixed size.
- Slices are flexible and more commonly used.
- Use `append()` and `len()` correctly.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Functions and Reusable Automation

07 VERIQTA

1. Why Functions Matter

- Functions help you organize your code.
- They let you reuse logic instead of writing the same code multiple times.
- Make your automation easier to maintain, read and test.
- In DevOps, functions are used for tasks like checking services, reading configurations and calling APIs.

2. Function Syntax and Parameters

```
package main

func greet(name string) {
    fmt.Println("Hello, " + name)
}

func main() {
    greet("DevOps Engineer")
}
```

- name `string` is a parameter.
- You can pass values to functions.
- Parameters let you reuse the same function with different inputs.

3. Return Values

```
func add(a int, b int) int {
    return a + b
}

func main() {
    result := add(5, 3)
    fmt.Println("Sum:", result)
}
```

- The function returns an integer using `return`.
- You can store the returned value in a variable.
- Return values allow functions to provide results to the caller.

4. Multiple Return Values

```
func divide(a int, b int) (int, error) {
    if b == 0 {
        return 0, fmt.Errorf("cannot divide by zero")
    }
    return a / b, nil
}
```

- A function can return multiple values.
- Commonly used to return a result and an error.
- This is widely used in DevOps automation.

5. Function Scope

```
func example() {
    message := "inside function" // local scope
    fmt.Println(message)
}

func main() {
    example()
    // fmt.Println(message) // Error: not defined
}
```

- Variables created inside a function have local scope.
- They cannot be accessed outside the function.
- This keeps your code clean and prevents unwanted changes.

6. Breaking Large Tasks into Smaller Functions

```
func readConfig(path string) string {
    // read configuration file
    return "config-data"
}

func checkService(name string) bool {
    // check if service is running
    return true
}

func main() {
    config := readConfig("app.yaml")
    fmt.Println("Config:", config)
    status := checkService("nginx")
    fmt.Println("Nginx running:", status)
}
```

- Split your code into small functions with a clear purpose.
- Each function does one thing.
- This makes your automation easier to maintain, test and extend.

7. Practical Example: Reusable Server Health-Check Function

```
import (
    "fmt"
    "net/http"
    "time"
)

func checkServer(url string) (int, error) {
    client := http.Client{Timeout: 5 * time.Second}
    resp, err := client.Get(url)
    if err != nil {
        return 0, err
    }
    defer resp.Body.Close()
    return resp.StatusCode, nil
}
```

- This function makes an HTTP request to a server and returns the status code.
- It also returns an error if the request fails.
- You can reuse this function to check multiple services like nginx, api or database.

Example Usage

```
func main() {
    code, err := checkServer("https://example.com")
    if err != nil {
        fmt.Println("Error:", err)
        return
    }
    fmt.Println("Status Code:", code)
}
```

8. DevOps Connection



- In DevOps, you often reuse the same logic to check services, read configurations, call APIs or process data.
- Functions help you avoid duplicating code.
- This makes your automation more reliable, easier to update and easier to troubleshoot.

9. Junior Engineer Tip

- Write small, focused functions.
- Give your functions clear and meaningful names.
- Reuse functions instead of copying and pasting code.
- Always handle errors in your functions.

10. Key Takeaways

- ✓ Functions let you reuse logic.
- ✓ You can pass parameters and return values.
- ✓ Functions can return multiple values.
- ✓ Variables inside a function have local scope.
- ✓ Break large automation into smaller functions.
- ✓ Reusable functions help you write maintainable DevOps tooling.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Structs, Methods, and Modeling Infrastructure 08

1. What is a Struct?

- A struct is a custom data type that groups related data together.
- It represents a real-world object (e.g. server, service, deployment).
- Helps you model infrastructure data in a structured way.
- Makes your code easier to read, maintain and extend.

3. Accessing and Modifying Fields

```
package main
import "fmt"
type Server struct {
    Name      string
    Port      int
    Environment string
}
func main() {
    s := Server{Name: "api-01", Port: 8080, Environment: "dev"}
    fmt.Println(s.Name)      // api-01
    fmt.Println(s.Port)      // 8080
    s.Environment = "staging" // modify field
    fmt.Println(s.Environment) // staging
```

- Access fields using the dot (.) operator.
- You can read and modify fields.

4. Methods

```
package main
import "fmt"
type Server struct {
    Name string
    Port int
}
// Method with value receiver
func (s Server) Info() string {
    return fmt.Sprintf("%s:%d", s.Name, s.Port)
}
func main() {
    s := Server{"web-01", 8080}
    fmt.Println(s.Info()) // web-01:8080
```

- A method is a function that belongs to a struct.
- It uses a special receiver (s Server) before the function name.
- Methods let you add behavior to your data.

6. Practical Example: Model and Check a Server

```
package main
import "fmt"
type Server struct {
    Name      string
    IP         string
    Port      int
    Status     string
}
// Method to check health (simple example)
func (s *Server) IsHealthy() bool {
    return s.Status == "up"
}
func (s Server) Details() string {
    return fmt.Sprintf("%s | %s | %d | %s", s.Name, s.IP, s.Port, s.Status)
}
func main() {
    s := Server{"web-01", "192.168.1.10", 8080, "up"}
    fmt.Println(s.Details()) // web-01 | 192.168.1.10 | 8080 | up
    fmt.Println("Healthy:", s.IsHealthy()) // Healthy: true
```

- Model a server with relevant fields.
- Use methods to add behavior.
- Keep related data and logic together.
- Makes your code cleaner and easier to reuse.

9. Junior Engineer Tip



- Use structs to model real infrastructure objects such as servers, services and deployments.
- This helps you write clean, maintainable and reusable automation.

2. Fields and Creating Struct Values

```
package main
import "fmt"
// Define a struct
type Server struct {
    Name      string
    IP         string
    Port      int
    Environment string
}
// Create a struct value
func main() {
    s := Server{
        Name:      "web-01",
        IP:        "192.168.1.10",
        Port:      8080,
        Environment: "prod",
    }
    fmt.Println(s)
}
```

- Fields store different types of data.
- Use the struct name and {} to create a value.
- Each field has a name and a type.

Output (example)

```
{web-01 192.168.1.10
8080 prod}
```

5. Pointer Receivers (Introductory)

```
package main
import "fmt"
type Server struct {
    Name      string
    Status     string
}
// Method with pointer receiver
func (s *Server) SetStatus(status string) {
    s.Status = status
}
func main() {
    s := Server{Name: "web-01", Status: "down"}
    fmt.Println(s.Status) // down
    s.SetStatus("up")
    fmt.Println(s.Status) // up
```

- Pointer receivers allow methods to modify the original struct.
- Use *StructName to receive a pointer.
- Useful when you need to change field values inside a method.

7. Why Structured Models Are Better



Groups related data (e.g. server details) in one place.



Easier to manage, read and maintain than unrelated variables.



Helps you build reusable functions and methods.



Common in real DevOps tools and infrastructure automation.

8. Connection to Larger Infrastructure Tools



Kubernetes
(models Pods,
Deployments, etc.)



Terraform
(uses structured
configurations)



Docker
(works with
structured data)



Prometheus
(uses structured
metrics data)

10. Key Takeaways

- ✓ A struct groups related data.
- ✓ Fields store values of different types.
- ✓ You can create, access and modify fields.
- ✓ Methods add behavior to structs.
- ✓ Pointer receivers let you modify the original struct.
- ✓ Use structs to model infrastructure for cleaner and more maintainable code.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



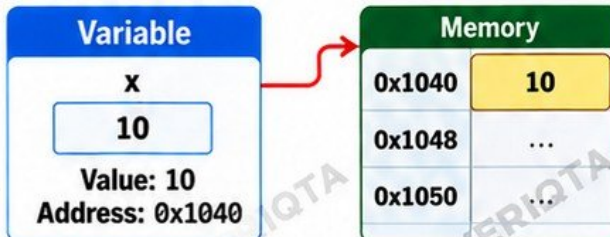
@veriqta

Pointers Without the Confusion

09 VERIQTA

1. Values and Memory

- Variables store values in memory.
- Each variable has a memory address.
- A value is the actual data (e.g. 10, "web", true).
- A pointer stores the memory address of a value, not the value itself.



2. What a Pointer Represents

- A pointer holds the memory address of a value.
- The & operator gives you the address of a variable.
- The * operator dereferences the address to get the value.

```
package main
import "fmt"
func main() {
    x := 10
    fmt.Println("Value:", x) // 10
    fmt.Println("Address:", &x) // e.g. 0x1040
    p := &x // p stores address of x
    fmt.Println("Pointer p:", p) // 0x1040
    fmt.Println("Value using *p:", *p) // 10
}
```

3. Passing Values vs References

Pass by Value

- A copy of the value is passed.
- Changes inside the function do not affect the original variable.

```
func increment(n int) {
    n = n + 1
}

func main() {
    x := 5
    increment(x)
    fmt.Println(x) // 5 (no change)
}
```

Pass by Reference (Pointer)

- The memory address is passed.
- Changes inside the function affect the original variable.

```
func increment(n *int) {
    *n = *n + 1
}

func main() {
    x := 5
    increment(&x)
    fmt.Println(x) // 6 (changed)
}
```

4. When Mutation Matters

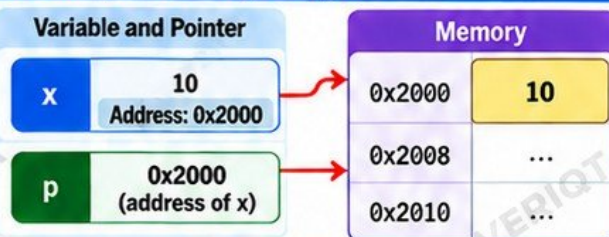
- Use pointers when you need to modify the original data inside a function.
- Common for large structs (to avoid copying).
- Used in APIs where functions need to update configuration, status or other data.

```
type Server struct {
    Name string
    Status string
}

func updateStatus(s *Server, status string) {
    s.Status = status // modifies the original struct
}

func main() {
    srv := Server{Name: "web-01", Status: "down"}
    updateStatus(&srv, "up")
    fmt.Println(srv.Status) // up
}
```

5. Simple Memory Diagram



- x stores the value 10.
- p stores the address of x (0x2000).
- Using *p gives you the value at 0x2000, which is 10.
- Using &x gives you the address 0x2000.

6. Why Pointers Appear in Go APIs

- Go's standard library and many infrastructure tools use pointers.
- Pointers allow functions to modify data (e.g. setting fields).
- They avoid unnecessary copying of large structures.
- Common in Kubernetes client libraries, configuration objects, and system APIs.

Examples in real tools:

 Kubernetes client-go uses pointers for object configuration.

 Docker APIs use pointers in Go structures.

 Terraform providers use pointers to configure resources.

 Many Go APIs use pointers to return results and errors.

7. Common Mistake

- Using pointers when value semantics are sufficient.
- Making code harder to read with unnecessary pointers.
- For simple types (int, string, bool), values are usually fine.
- Only use pointers when you need to modify the data or avoid copying large structures.

9. Junior Engineer Tip

Start with values, then learn pointers. Use pointers when you need to modify data or when working with large structures in real DevOps projects and APIs.

8. Key Takeaways

- A pointer stores a memory address, not a value.
- Use & to get the address and * to access the value.
- Pass by value creates a copy, pass by reference allows modification.
- Use pointers when mutation matters or when working with large data.
- Pointers appear frequently in Go APIs, including cloud-native tools.
- Keep code simple and use pointers only when needed.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Errors and Defensive Automation

10 VERIQTA

1. Go's Explicit Error Model

- Go does not use exceptions.
- Functions return an error as a value.
- You must check the error and handle it.
- This makes failures visible and prevents silent errors in automation.

```
package main
import (
    "fmt"
    "os"
)

func main() {
    file, err := os.Open("config.yaml")
    if err != nil {
        fmt.Println("Error:", err)
        return
    }
    file.Close()
}
```

2. The error Type

- The `error` type is a built-in interface in Go.
- It represents a failure with a message.
- A function often returns a value and an error.

```
func readFile(path string) (string, error) {
    data, err := os.ReadFile(path)
    if err != nil {
        return "", err
    }
    return string(data), nil
}
```

- The function returns both a value and an error.
- If successful, error is `nil`.
- If it fails, error contains the reason.

3. if err != nil

- Always check the error after a function call.
- Do not assume the operation succeeded.

```
data, err := os.ReadFile("app.yaml")
if err != nil {
    fmt.Println("Failed to read file:", err)
    return
}
```

- If `err` is not `nil`, something went wrong.
- Handle the error before using the value.

4. Returning Errors

- Your own functions should also return errors.
- This allows the caller to handle the failure.

```
func loadConfig(path string) (map[string]string, error) {
    data, err := os.ReadFile(path)
    if err != nil {
        return nil, err
    }
    // parse data (simplified)
    config := map[string]string{"env": "prod"}
    return config, nil
}
```

5. Adding Context to Errors

- Use `fmt.Errorf` to add useful context to an error.
- This helps you understand what went wrong.

```
func loadConfig(path string) (map[string]string, error) {
    data, err := os.ReadFile(path)
    if err != nil {
        return nil, fmt.Errorf("failed to read file %s: %w",
            path, err)
    }
    return map[string]string{"env": "prod"}, nil
}
```

- `%w` wraps the original error, so you keep the root cause while adding your own message.

6. errors Package and Useful Functions

- The `errors` package provides helper functions.
- Use `errors.Is` to check for specific errors.
- Use `errors.New` to create a simple error.

```
import (
    "errors"
    "fmt"
    "os"
)

if errors.Is(err, os.ErrNotExist) {
    fmt.Println("File does not exist")
}
```

Common functions

- `errors.New()` creates a new error.
- `fmt.Errorf()` adds context.
- `errors.Is()` checks for a specific error.
- `errors.As()` checks for a specific type of error.

7. Why Automation Must Not Silently Ignore Failures

- Ignoring errors can hide real problems.
- This can cause your automation to continue with incorrect data, leading to bigger failures.

Bad Pattern

- ```
_ = riskyOperation()
```
- The error is ignored.
  - You lose important information.
  - It makes troubleshooting difficult.

### Good Pattern

- ```
err := riskyOperation()
if err != nil {
    log.Fatalf("operation failed: %v", err)
}
```
- Always check and handle the error.
 - Log useful information.
 - Decide whether to retry, exit or continue with fallback logic.

8. Troubleshooting Mindset

- Read the full error message.
- Preserve useful error information when returning it.
- Add your own context to make the error clear.
- Use logs to capture the error with relevant details such as file name, resource, environment or operation.

9. Junior Engineer Tip

- Always check for errors.
- Use meaningful error messages.
- Do not ignore errors.
- Make your automation fail safely and with clear logs.

10. Key Takeaways

- Go uses an explicit error model.
- Always check if `err != nil`.
- Return errors from your own functions.
- Add context using `fmt.Errorf`.
- Do not silently ignore failures.
- Read and preserve useful error information.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Files, Directories, and Configuration

11



1. Read Files

- Use `os.ReadFile()` to read the entire file.
- It returns the content as a byte slice.
- Convert the bytes to a string.

```
package main
import (
    "fmt"
    "os"
)

func main() {
    data, err := os.ReadFile("config.yaml")
    if err != nil {
        fmt.Println("Error:", err)
        return
    }
    fmt.Println(string(data))
}
```

What it does:

- Reads the content of config.yaml from disk.
- Returns the file content as a string.
- Useful for reading application configuration files.

2. Write Files

- Use `os.WriteFile()` to create or overwrite a file.
- Provide the content and file permissions.

```
package main
import (
    "os"
)

func main() {
    content := "app: nginx\nenv: prod\nport: 8080\n"
    err := os.WriteFile("app.yaml", []byte(content), 0644)
    if err != nil {
        fmt.Println("Error:", err)
        return
    }
    fmt.Println("File written successfully")
}
```

0644 permissions mean:

- Owner can read and write.
- Group and others can read.

- Use different permissions for sensitive files.

3. File Permissions

- Use `os.Chmod()` to change file permissions.
- Permissions control who can read, write or execute files.

```
package main
import "os"

func main() {
    err := os.Chmod("app.yaml", 0600) // owner read/write only
    if err != nil {
        println("Error:", err)
    }
}
```

Common permissions:

- 0644 – owner read/write, others read
- 0600 – owner read/write only (good for sensitive files)
- 0755 – owner read/write/execute, others read/execute

4. Check Whether Files Exist

- Use `os.Stat()` to check if a file or directory exists.
- It returns an error if the file does not exist.

```
package main
import (
    "fmt"
    "os"
)

func main() {
    _, err := os.Stat("app.yaml")
    if err != nil {
        if os.IsNotExist(err) {
            fmt.Println("File does not exist")
        } else {
            fmt.Println("Error:", err)
        }
    } else {
        fmt.Println("File exists")
    }
}
```

Useful checks:

- `os.IsNotExist(err)` checks if the file does not exist.
- `os.IsPermission(err)` checks for permission errors.
- Can also be used for directories.

5. Work with Paths Safely

- Use the `path/filepath` package to work with file paths.
- It handles different operating systems safely.

```
package main
import (
    "fmt"
    "path/filepath"
)

func main() {
    base := "/etc"
    file := "nginx/nginx.conf"
    fullPath := filepath.Join(base, file)
    fmt.Println("Full path:", fullPath)

    dir := filepath.Dir(fullPath)
    fmt.Println("Directory:", dir)

    name := filepath.Base(fullPath)
    fmt.Println("File name:", name)
}
```

Common functions:

- `filepath.Join()` combines paths safely.
- `filepath.Dir()` gets the directory.
- `filepath.Base()` gets the file name.
- Works on Linux, Windows and macOS.

6. Practical Example: Read a Configuration File

- Read an application configuration file (YAML or simple key-value).
- This example reads a simple key-value configuration file.

```
package main
import (
    "bufio"
    "fmt"
    "os"
    "strings"
)

func main() {
    file, err := os.Open("app.conf")
    if err != nil {
        fmt.Println("Error:", err)
        return
    }
    defer file.Close()
    scanner := bufio.NewScanner(file)
    for scanner.Scan() {
        line := scanner.Text()
        if strings.Contains(line, "=") {
            parts := strings.SplitN(line, "=", 2)
            key := strings.TrimSpace(parts[0])
            value := strings.TrimSpace(parts[1])
            fmt.Printf("%s = %s\n", key, value)
        }
    }
}
```

Example config file (app.conf):

```
app=nginx
env=prod
port=8080
log_level=info
```

Output:

```
app = nginx
env = prod
port = 8080
log_level = info
```

7. Security Reminder

- Never overwrite important configuration files blindly.
- Always create a backup before making changes.
- Use proper file permissions for sensitive files.
- Validate content before writing to a file.

8. Verify Before and After Changes

- Check the file content before making changes.
- Confirm the changes after writing.
- Use `os.Stat()` to verify file size and permissions.
- Log what was changed for troubleshooting.

9. Junior Engineer Tip

- Always test file operations in a safe environment first.

10. Key Takeaways

- ✓ Use `os` and `path/filepath`.
- ✓ Always handle errors.
- ✓ Check file existence.
- ✓ Use correct permissions.
- ✓ Never ignore file operations.
- ✓ Verify changes.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



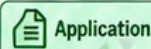
@veriqta

Environment Variables and Secrets

12 VERIQTA

1. What are Environment Variables?

- Environment variables store configuration values outside your code.
- They separate code from environment-specific settings (e.g. dev, staging, prod).
- Common in CI/CD pipelines, containers, and cloud platforms.
- Makes your application more flexible, secure and easier to deploy.



Application

Environment Variables

API_URL
DB_HOST
DB_PASSWORD
PORT
ENV=prod

2. Using os.Getenv() to Read a Variable

- `os.Getenv()` returns the value of an environment variable.
- If the variable does not exist, it returns an empty string.

```
package main
import (
    "fmt"
    "os"
)
func main() {
    apiURL := os.Getenv("API_URL")
    fmt.Println("API URL:", apiURL)
}
```

Example Output

API URL:
https://api.example.com

- If the variable is not set, the output will be an empty string.

3. Using os.LookupEnv()

- `os.LookupEnv()` returns the value and a boolean indicating whether the variable exists.
- Useful when you need to check if a variable is set.

```
package main
import (
    "fmt"
    "os"
)
func main() {
    token, exists := os.LookupEnv("API_TOKEN")
    if exists {
        fmt.Println("Token exists:", token)
    } else {
        fmt.Println("API_TOKEN is not set")
    }
}
```

Key Points:

- Returns (value, true) if the variable exists.
- Returns ("", false) if the variable does not exist.
- Helps you handle missing variables properly.

4. Defaults vs Required Values

Use a Default Value

- Use a default value when the variable is optional.

```
func getEnv(key, defaultValue string) string {
    if value := os.Getenv(key); value != "" {
        return value
    }
    return defaultValue
}
func main() {
    port := getEnv("PORT", "8080")
    fmt.Println("Port:", port)
}
```

Require a Value

- Return an error or exit when a required variable is missing.

```
func getRequiredEnv(key string) string {
    value, exists := os.LookupEnv(key)
    if !exists || value == "" {
        log.Fatalf("%s is required", key)
    }
    return value
}
func main() {
    apiURL := getRequiredEnv("API_URL")
    fmt.Println("API URL:", apiURL)
}
```

5. Practical Example: Read API Endpoint and Token Safely

- Read an API endpoint and token from environment variables.
- Use a default for the endpoint and require the token.

```
package main
import (
    "fmt"
    "log"
    "os"
)
func getEnv(key, defaultValue string) string {
    if value := os.Getenv(key); value != "" {
        return value
    }
    return defaultValue
}
func getRequiredEnv(key string) string {
    value, exists := os.LookupEnv(key)
    if !exists || value == "" {
        log.Fatalf("%s is required", key)
    }
    return value
}
func main() {
    apiURL := getEnv("API_URL", "https://api.example.com")
    apiToken := getRequiredEnv("API_TOKEN")
    fmt.Println("API URL:", apiURL)
    fmt.Println("Using token with length:", len(apiToken)) // do not print token
}
```

Example .env file

API_URL=https://api.example.com
API_TOKEN=abcd1234
ENV=prod

Program Output

API URL: https://api.example.com
Using token with length: 8

Notes:

- Never print the actual token in logs.
- Use a default for optional values.
- Require important values such as tokens, passwords or API keys.

6. Why Secrets Should Not Be Hardcoded

- Hardcoding secrets (API keys, passwords, tokens) in code is a serious security risk.
- Secrets can be exposed in Git repositories, logs and container images.
- Always use environment variables or a secret management solution.

Bad Pattern ❌

```
func main() {
    apiKey := "sk-123456789"
    // never do this
    fmt.Println(apiKey)
}
```

Good Pattern ✅

```
func main() {
    apiKey := os.Getenv("API_KEY")
    // handle missing value
    _ = apiKey
}
```

7. Where Environment Variables Are Commonly Used

- CI/CD pipelines (GitHub Actions, GitLab CI, Jenkins, etc.)
- Containers (Docker, Kubernetes)
- Cloud platforms (AWS, Azure, GCP)
- Application configuration (different values for dev, staging, prod)



8. Security Connection to CI/CD and Kubernetes Secrets

CI/CD Secret Stores



Kubernetes Secrets

Store secrets such as API keys, database passwords and tokens securely.

- Environment variables are often overwritten from secret stores.
- Kubernetes Secrets can be mounted as environment variables or files.
- Keep secrets out of your code, repositories and logs.

11. Junior Engineer Tip

- Always check if required environment variables are set before running your application.
- Use clear error messages when a variable is missing.
- Keep your configuration flexible so the same code can run in dev, staging and prod.

9. Verification Before and After

- Check that the variable is set: `value, exists := os.LookupEnv("API_TOKEN")`
- Verify the value looks correct (e.g. non-empty, correct format).
- Add logs (without printing the secret).
- Test in your local environment and in CI/CD.

10. Key Takeaways

- Use `os.Getenv()` or `os.LookupEnv()`.
- Use defaults for optional values.
- Require important variables.
- Never hardcode secrets.
- Environment variables are essential in CI/CD and containers.
- Use secret stores for sensitive data.
- Verify values before using them.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

JSON, YAML, and Configuration Data 13

1. Why DevOps Engineers Encounter Structured Data

- Configuration files (JSON, YAML).
- API requests and responses.
- Infrastructure tools (Kubernetes, Terraform, Docker).
- Application configuration and environment settings.
- Logs, monitoring data, and service definitions.
- Structured data makes automation consistent, scalable and easy to integrate.

2. JSON Concepts

- JSON (JavaScript Object Notation) is a lightweight format to store structured data.
- Used widely in APIs, configuration files and cloud tools.
- Consists of key-value pairs, arrays and nested objects.

Example JSON

```
{  "name": "web-01",  "port": 8080,  "env": "prod",  "tags": ["web", "linux"],  "enabled": true}
```

JSON Data Types

- String: "text"
- Number: 8080
- Boolean: true / false
- Array: [1, 2, 3]
- Object: { "key": "value" }

3. Go Structs and JSON Tags

- Use structs to represent JSON data in Go.
- Add tags such as `json:"name"` to map JSON fields to struct fields.

```
package main
import (
    "encoding/json"
    "fmt"
)

type Service struct {
    Name string `json:"name"`
    Port int    `json:"port"`
    Env  string `json:"env"`
    Tags []string `json:"tags"`
    Enabled bool `json:"enabled"`
}
```

Key Points:

- Tags tell Go how to map JSON fields to struct fields.
- Field names in JSON and Go can be different.
- `json:"name"` maps the JSON key "name" to the Go field Name.

4. Encoding and Decoding JSON

- Use `json.Marshal()` to convert Go data to JSON.
- Use `json.Unmarshal()` to convert JSON to Go structs.

Decode JSON (Unmarshal)

```
package main
import (
    "encoding/json"
    "fmt"
)

func main() {
    data := []byte(`{
        "name": "web-01",
        "port": 8080,
        "env": "prod"
    }`)
    var s Service
    err := json.Unmarshal(data, &s)
    if err != nil {
        fmt.Println("Error:", err)
        return
    }
    fmt.Printf("%+v\n", s)
}
```

Encode JSON (Marshal)

```
package main
import (
    "encoding/json"
    "fmt"
)

func main() {
    s := Service{Name: "api-01",
        Port: 9090, Env: "dev",
        Tags: []string{"api", "linux"},
        Enabled: true}
    data, err := json.MarshalIndent(s,
        "", " ")
    if err != nil {
        fmt.Println("Error:", err)
        return
    }
    fmt.Println(string(data))
}
```

5. Reading API Responses

- Most APIs return data in JSON format.
- Decode the JSON response into a Go struct.

```
package main
import (
    "encoding/json"
    "fmt"
    "net/http"
)

type APIResponse struct {
    Status string `json:"status"`
    Version string `json:"version"`
}

func main() {
    resp, err := http.Get("https://api.example.com/info")
    if err != nil {
        fmt.Println("Request error:", err)
        return
    }
    defer resp.Body.Close()
    var result APIResponse
    err = json.NewDecoder(resp.Body).Decode(&result)
    if err != nil {
        fmt.Println("Decode error:", err)
        return
    }
    fmt.Printf("Status: %s, Version: %s\n", result.Status, result.Version)
}
```

Output (example)

```
Status: ok
Version: v1.2.0
```

6. YAML Ecosystem Context

- YAML is commonly used in:
 - Kubernetes manifests
 - CI/CD pipelines (GitHub Actions, GitLab CI)
 - Configuration files (applications, infrastructure tools)
- Easier for humans to read than JSON

Example YAML

```
name: web-01
port: 8080
env: prod
tags:
  - web
  - linux
enabled: true
```

7. Parse YAML in Go

- Use a YAML library such as `gopkg.in/yaml.v3`.
- Works similarly to JSON with structs and tags.

```
package main
import (
    "fmt"
    "gopkg.in/yaml.v3"
)

type Config struct {
    Name string `yaml:"name"`
    Port int    `yaml:"port"`
    Env  string `yaml:"env"`
}

func main() {
    data := []byte(`name: api-01
port: 9090
env: dev`)
    var c Config
    err := yaml.Unmarshal(data, &c)
    if err != nil {
        fmt.Println("Error:", err)
        return
    }
    fmt.Printf("%+v\n", c)
}
```

8. Practical Example: Parse Service Configuration

- Read a JSON or YAML configuration file for services.
- Convert it to Go structs.
- Validate the data before using it.

```
func loadServiceConfig(path string) ([]Service, error) {
    data, err := os.ReadFile(path)
    if err != nil {
        return nil, fmt.Errorf("failed to read file: %w", err)
    }
    var services []Service
    err = json.Unmarshal(data, &services) // or yaml.Unmarshal
    if err != nil {
        return nil, fmt.Errorf("invalid configuration: %w", err)
    }
    return services, nil
}
```

Example config file (JSON)

```
{
  "name": "web-01", "port": 8080,
  "env": "prod",
  "name": "api-01", "port": 9090,
  "env": "dev"
}
```

Output (example)

```
Loaded 2 services
web-01 (8080) prod
api-01 (9090) dev
```

9. Common Failures and Validation

- Malformed JSON or YAML (syntax errors).
- Missing required fields.
- Incorrect data types (string instead of number).
- Assuming the file or API response is always correct.

Example Error (malformed JSON)

```
Error: invalid character '}'
looking for beginning of object key string
```

Validate Input

- ✓ Check for required fields.
- ✓ Validate data types.
- ✓ Use sensible defaults.
- ✓ Return clear error messages.

10. Security Connection to CI/CD and Kubernetes Secrets

- Do not store secrets in JSON or YAML files in your code repository.
- Use environment variables or secret stores.
- In CI/CD: GitHub Actions Secrets, GitLab CI Variables.
- In Kubernetes: Secrets (mounted as environment variables or files).
- Keep configuration flexible so the same code works in dev, staging and production.



11. Junior Engineer Tip

- Always validate configuration data before using it.
- Use clear error messages.
- Keep configuration flexible and environment-agnostic.

12. Key Takeaways

- ✓ JSON and YAML are used a lot in DevOps.
- ✓ Use clear naming and map values.
- ✓ Never assume configuration is correct.
- ✓ Use secret stores for sensitive data.
- ✓ Keep automation flexible and secure.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Running Linux Commands from Go

14 VERIQTA

1. What is os/exec?

- The os/exec package allows you to run external commands from your Go program.
- You can execute Linux commands, capture their output and handle errors.
- Useful for automation, system administration and DevOps tasks.

```
package main
import (
    "fmt"
    "os/exec"
)

func main() {
    cmd := exec.Command("echo", "Hello from Go")
    out, err := cmd.Output()
    if err != nil {
        fmt.Println("Error:", err)
        return
    }
    fmt.Println(string(out))
}
```

Key Points:

- exec.Command() creates a command to run.
- You pass the command and arguments separately (safer).
- Output() runs the command and returns the standard output.
- It returns an error if the command fails.

2. exec.Command() Syntax

- exec.Command(name, arg1, arg2, ...)
- The first argument is the command.
- The following arguments are passed to the command.
- Do not include the full command as a single string (e.g. "ls -l"). Pass each part separately.

Example: List Files

```
package main
import (
    "fmt"
    "os/exec"
)

func main() {
    cmd := exec.Command("ls", "-l", "/tmp")
    out, err := cmd.Output()
    if err != nil {
        fmt.Println("Error:", err)
        return
    }
    fmt.Println(string(out))
}
```

Command Parts

Command:
ls

Argument 1:
-l

Argument 2:
/tmp

Each argument is passed separately, which prevents command injection.

3. Capturing Output and Handling Errors

- Output() captures standard output (stdout) only.
- CombinedOutput() captures both stdout and stderr.
- If the command fails, err will contain the exit error and the output may still have useful information.

Capture Only Standard Output

```
cmd := exec.Command("uname", "-a")
out, err := cmd.Output()
if err != nil {
    fmt.Println("Error:", err)
    return
}
fmt.Println("Output:")
fmt.Println(string(out))
```

Capture Stdout and Stderr

```
cmd := exec.Command("cat", "/etc/hosts")
out, err := cmd.CombinedOutput()
if err != nil {
    fmt.Println("Command failed:", err)
    fmt.Println("Output:")
    fmt.Println(string(out))
    return
}
fmt.Println(string(out))
```

Typical Output:

Linux ip-172-31-0-1 5.15.0-1051-aws
#56-Ubuntu SMP ... x86_64 x86_64 ...

If the command fails:

Command failed: exit status 1
Output: cat: /etc/hosts: No such file or directory

4. Practical Example: Check systemctl

- This example checks the status of a service using systemctl.
- It captures the output and handles failures.

```
package main
import (
    "fmt"
    "os/exec"
)

func main() {
    service := "nginx"
    cmd := exec.Command("systemctl", "is-active", service)
    out, err := cmd.CombinedOutput()
    if err != nil {
        fmt.Printf("Service %s is not active\n", service)
        fmt.Println("Error:", err)
        fmt.Println("Output:", string(out))
        return
    }
    fmt.Printf("Service %s is active\n", service)
    fmt.Println("Output:", string(out))
}
```

Example Output (active):

Service nginx is active
Output: active

Example Output (failed):

Service nginx is not active
Error: exit status 3
Output: inactive

5. Why Shelling Out is Sometimes Useful

- Allows you to use existing Linux tools and utilities.
- Useful for system administration, monitoring and quick automation.
- Helps you leverage powerful tools like systemctl, cat, grep, awk, etc.

Common Use Cases:

			
Check service status	Read system information	Run diagnostic tools	Collect logs and metrics

6. Native Go Libraries Can Be Safer

- Many tasks have native Go libraries (e.g. read files, make HTTP requests, manage processes).
- Native libraries are more secure, portable and give you better error handling.
- Shelling out should be used only when necessary.

Examples of native alternatives:

- Read files: os and ioutil (or os.ReadFile)
- HTTP requests: net/http
- Manage processes: os and syscall
- Work with services: use Go libraries or APIs instead of systemctl when possible

7. Security Warning

- Never construct commands using untrusted input (e.g. user input).
- Do not use shell execution (e.g. exec.Command("bash", "-c", ...)) with untrusted data.
- Pass command arguments separately.
- Validate and sanitize any input used in commands.
- Command injection can lead to serious security issues.

8. Common Mistakes

- Using a single string: exec.Command("ls -l") (wrong).
- Ignoring errors from command execution.
- Not checking stderr output.
- Allowing untrusted input to be used in commands.

9. Junior Engineer Tip

- Always check the error and read the output.
- Use CombinedOutput() when you need full error details.
- Test commands manually in the terminal first.
- Keep your automation secure and avoid unnecessary shelling out.

10. Key Takeaways

- Use os/exec to run Linux commands from Go.
- Use exec.Command() with separate arguments.
- Capture output and handle errors properly.
- Shelling out is useful but use native Go libraries when possible.
- Never use untrusted input in commands.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



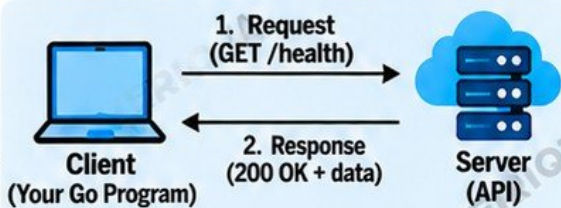
@veriqta

HTTP and REST APIs

15 VERIQTA

1. HTTP Mental Model

- HTTP is a protocol for communication between a client and a server.
- The client sends a request and the server returns a response.
- REST APIs use HTTP methods (GET, POST, etc.) to perform operations on resources.



2. GET Requests in Go

- Use `http.Get()` for simple requests.
- For more control (headers, timeouts, etc.), use `http.Client`.

```
package main
import (
    "fmt"
    "net/http"
    "io/ioutil"
)

func main() {
    resp, err := http.Get("https://api.example.com/health")
    if err != nil {
        fmt.Println("Error:", err)
        return
    }
    defer resp.Body.Close()
    body, err := ioutil.ReadAll(resp.Body)
    if err != nil {
        fmt.Println("Error reading body:", err)
        return
    }
    fmt.Println("Status:", resp.StatusCode)
    fmt.Println("Response:", string(body))
}
```

What Happens?

- 1 Send a GET request to the API endpoint.
- 2 Receive a response from the server.
- 3 Check the status code and read the response body.

3. Status Codes

- Status codes indicate the result of the request.
- Common status codes:

Code	Meaning
200	OK (success)
201	Created (success)
400	Bad Request (client error)
401	Unauthorized (client error)
404	Not Found (client error)
500	Internal Server Error (server error)
503	Service Unavailable (server error)

4. Response Bodies

- The response body contains the data returned by the server.
- Often in JSON format.

Example JSON Response

```
{
  "status": "ok",
  "service": "api",
  "version": "1.2.0"
}
```

5. Headers

- Headers provide additional information about the request and response.
- You can set headers in your request and read headers from the response.

```
package main
import (
    "net/http"
)

func main() {
    req, _ := http.NewRequest("GET", "https://api.example.com/health", nil)
    req.Header.Set("Authorization", "Bearer mytoken")
    resp, _ := http.DefaultClient.Do(req)
    contentType := resp.Header.Get("Content-Type")
    fmt.Println("Content-Type:", contentType)
}
```

6. Timeouts

- Always set a timeout to avoid hanging requests.
- Use `http.Client` with a timeout.

```
package main
import (
    "net/http"
    "time"
)

func main() {
    client := &http.Client{
        Timeout: 10 * time.Second,
    }
    resp, err := client.Get("https://api.example.com/health")
    if err != nil {
        // Handle error
    }
}
```

7. Error Handling

- Always check for errors.
- Handle network errors, timeouts, and non-200 status codes.
- Do not assume a successful connection means a healthy application.

```
package main
import (
    "net/http"
    "fmt"
)

func main() {
    resp, err := client.Get(url)
    if err != nil {
        fmt.Println("Request error:", err)
        return
    }
    if resp.StatusCode != http.StatusOK {
        fmt.Printf("Unexpected status: %d\n", resp.StatusCode)
    }
}
```

8. Practical Example: Query a Health Endpoint

- This example checks the health of a service and parses the JSON response.

```
package main
import (
    "encoding/json"
    "fmt"
    "net/http"
    "time"
)

type HealthResponse struct {
    Status string `json:"status"`
    Version string `json:"version"`
}

func main() {
    client := &http.Client{Timeout: 5 * time.Second}
    resp, err := client.Get("https://api.example.com/health")
    if err != nil {
        fmt.Println("Request error:", err)
        return
    }
    defer resp.Body.Close()
    if resp.StatusCode != http.StatusOK {
        fmt.Println("Unexpected status:", resp.StatusCode)
        return
    }
    var data HealthResponse
    err = json.NewDecoder(resp.Body).Decode(&data)
    if err != nil {
        fmt.Println("JSON parse error:", err)
        return
    }
    fmt.Printf("Service: %s, Version: %s\n", data.Status, data.Version)
}
```

9. Production Awareness

- A successful network connection does not automatically mean the application is healthy.
- The server might return an error status code or an unhealthy response body.
- Always check the status code and the response content.

Healthy Response

```
{
  "status": "ok",
  "version": "1.2.0"
}
```

Status Code: 200

Unhealthy Response

```
{
  "status": "error",
  "message": "database not available"
}
```

Status Code: 503

10. Common Use Cases

- ✓ Check service health
- ✓ Query cloud APIs (AWS, Kubernetes, etc.)
- ✓ Retrieve configuration data
- ✓ Monitor application status
- ✓ Integrate with third-party APIs

11. Security Considerations

- Never hardcode API keys or tokens.
- Use environment variables or secret stores.
- Validate API responses before using the data.
- Avoid logging sensitive information.
- Use HTTPS for secure communication.

12. Key Takeaways

- ✓ Understand the client → request → server → response model.
- ✓ Use `http.Client` with timeouts.
- ✓ Check status codes and handle errors.
- ✓ Parse JSON responses into Go structs.
- ✓ A successful network connection does not mean the application is healthy.
- ✓ Always follow security best practices.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



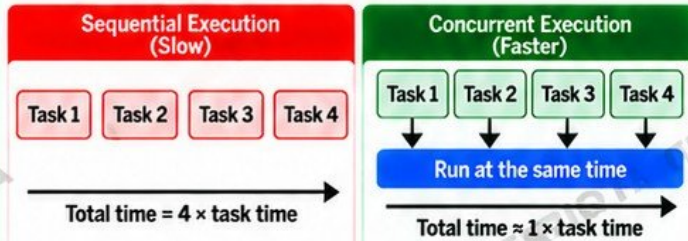
@veriqta

Concurrency for Infrastructure Tasks

16 VERIQTA

1. Why Sequential Automation Can Become Slow

- Running tasks one after another takes time.
- In infrastructure automation, you often need to check many resources (servers, APIs, services).
- Sequential execution waits for each task to finish before starting the next one.
- Concurrency allows multiple tasks to run at the same time, which makes automation faster and more efficient.



2. Goroutines and go function()

- A goroutine is a lightweight thread managed by Go.
- Use the go keyword to start a function as a goroutine.
- Goroutines run concurrently and are very cheap to create.

```
package main
import (
    "fmt"
    "time"
)

func worker(id int) {
    fmt.Printf("Worker %d starting\n", id)
    time.Sleep(2 * time.Second)
    fmt.Printf("Worker %d finished\n", id)
}

func main() {
    go worker(1)
    go worker(2)
    fmt.Println("Both workers started")
    time.Sleep(3 * time.Second)
}
```

Key Points:

- Use go function() to run a function concurrently.
- Goroutines run independently.
- The main function does not wait unless you use synchronization (e.g. WaitGroup).

3. Channels

- Channels let goroutines communicate and share data.
- Use make(chan type) to create a channel.
- Use <- to send and receive data.

```
package main
import "fmt"

func main() {
    ch := make(chan string)
    go func() {
        ch <- "Hello from goroutine"
    }()
    msg := <-ch
    fmt.Println(msg)
}
```

How Channels Work:



Key Points:

- Channels are typed (e.g. chan string).
- They are safe for communication between goroutines.
- They help you avoid shared memory issues.

4. Synchronization with sync.WaitGroup

- Use sync.WaitGroup to wait for multiple goroutines to finish.
- Add the number of goroutines, call Done() when each finishes, and use Wait() to block until all are complete.

```
package main
import (
    "fmt"
    "sync"
    "time"
)

func worker(id int, wg *sync.WaitGroup) {
    defer wg.Done()
    fmt.Printf("Worker %d starting\n", id)
    time.Sleep(2 * time.Second)
    fmt.Printf("Worker %d finished\n", id)
}

func main() {
    var wg sync.WaitGroup
    wg.Add(2)
    go worker(1, &wg)
    go worker(2, &wg)
    wg.Wait()
    fmt.Println("All workers finished")
}
```

How WaitGroup Works:

- 1 wg.Add(n)
Set the number of goroutines to wait for.
- 2 defer wg.Done()
Call when a goroutine finishes.
- 3 wg.Wait()
Blocks until all goroutines are done.

5. Practical Example: Concurrent Health Checks

- This example checks multiple HTTP endpoints concurrently.
- Each endpoint is checked in a separate goroutine.
- We use a channel to collect the results and a WaitGroup to wait for all checks to finish.

```
package main
import (
    "fmt"
    "net/http"
    "sync"
    "time"
)

type Result struct {
    URL    string
    Status string
    Error  string
}

func checkEndpoint(url string, ch chan Result, wg *sync.WaitGroup) {
    defer wg.Done()
    client := http.Client{Timeout: 5 * time.Second}
    resp, err := client.Get(url)
    if err != nil {
        ch <- Result{URL: url, Status: "error", Error: err.Error()}
    } else {
        defer resp.Body.Close()
        ch <- Result{URL: url, Status: resp.Status, Error: ""}
    }
}

func main() {
    urls := []string{
        "https://api.example.com/health",
        "https://google.com",
        "https://github.com",
    }
    var wg sync.WaitGroup
    ch := make(chan Result, len(urls))
    for _, url := range urls {
        wg.Add(1)
        go checkEndpoint(url, ch, &wg)
    }
    wg.Wait()
    for result := range ch {
        fmt.Println(result)
    }
}
```

Example Output

```
https://api.example.com/health
→ 200 OK
https://google.com
→ 200 OK
https://github.com
→ 200 OK
```

How It Works:

- Start a goroutine for each URL.
- Each goroutine checks the endpoint.
- Results are sent to a channel.
- WaitGroup waits for all goroutines.
- Print the results after all checks finish.

6. Race Conditions (Beginner Level)

- A race condition happens when multiple goroutines access and modify shared data at the same time.
- This can lead to unpredictable and incorrect results.
- Use channels or synchronization (e.g. mutexes) to avoid races.

Example (race condition):

```
package main
import "fmt"

func main() {
    counter := 0
    go func() { counter++ }()
    go func() { counter++ }()
    time.Sleep(1 * time.Second)
    fmt.Println("Counter:", counter) // unpredictable result
}
```

How to Fix:

- Use channels to share data.
- Use sync.Mutex for shared variables.
- Or design your code to avoid shared mutable state.

7. Do Not Confuse Concurrency with Unlimited Parallel Execution

- Concurrency means multiple tasks can make progress, but they are not always running at the exact same time.
- Go (and your system) may use a limited number of CPU cores.
- Too many goroutines can lead to high memory usage and reduced performance.
- Use concurrency wisely and consider rate limiting for external services.

8. Production Awareness



- A successful network connection does not automatically mean the application is healthy.
- Check the actual status code and response body.
- Validate that the service returns the expected data.
- Set reasonable timeouts and handle errors properly.

9. Junior Engineer Tip



- Start with simple concurrent tasks.
- Use WaitGroup to avoid premature exit.
- Always handle errors and check responses.
- Do not create too many goroutines at once.

10. Key Takeaways

- Use go function() to run tasks concurrently.
- Use channels for communication.
- Use sync.WaitGroup to wait for completion.
- Avoid race conditions.
- Do not confuse concurrency with unlimited parallelism.
- Use concurrency to make infrastructure automation faster and safer.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Context, Timeouts, and Cancellation

17 VERIQTA

1. Why Automation Must Have Limits

- External operations (APIs, databases, network calls) can hang, be slow, or become unresponsive.
- Without timeouts, your automation can run forever and block resources.
- Limits (timeouts and cancellation) make automation more reliable, predictable and safe.
- Important in production where network issues, service failures or overloads are common.

Without Timeouts (Problem)



With Timeouts (Safe)



2. context.Context in Go

- `context.Context` carries a deadline, cancellation signal and other request-scoped values.
- Used to control the lifetime of operations such as HTTP requests, database calls, and API calls.
- Helps you cancel work when it is no longer needed.

```
package main
import (
    "context"
    "fmt"
    "time"
)

func main() {
    ctx, cancel := context.
    WithTimeout(context.Background(),
        5*time.Second)
    defer cancel() // always cancel to free resources

    fmt.Println("Context deadline:",
        time.Now().Add(5*time.Second))
    _ = ctx
}
```

Key Points:

- Context can have a deadline (timeout).
- It can be cancelled manually.
- It is passed to functions that support context.
- When the context is done, operations should stop quickly.

3. Deadlines and Timeouts

- A deadline is a specific time when the operation must finish.
- A **timeout** is a duration from now (e.g. 5 seconds).
- Use `context.WithTimeout()` to set a timeout.
- Use `context.WithDeadline()` to set an exact deadline.

Timeout Example

```
ctx, cancel := context.WithTimeout(
    context.Background(),
    5*time.Second,
)
defer cancel()
// use ctx with an operation
```

Deadline Example

```
deadline := time.Now().Add(10 * time.Second)
ctx, cancel := context.WithDeadline(
    context.Background(),
    deadline,
)
defer cancel()
```

4. Cancellation

- You can cancel an operation manually using `cancel()`.
- Useful when a request is no longer needed (e.g. user cancelled, program shutting down, or another condition is met).

```
ctx, cancel := context.WithCancel(context.Background())

go func() {
    // simulate some work
    time.Sleep(2 * time.Second)
    cancel() // cancel the operation
}()

<-ctx.Done()
fmt.Println("Operation cancelled:", ctx.Err())
```

Common Cancellation Reasons:

- 1 User request cancelled.
- 2 Program is shutting down.
- 3 A dependent operation failed.
- 4 No longer needed.

5. HTTP Request with Timeout Example

- Use an HTTP client with a context and timeout to avoid hanging requests.
- If the server does not respond in time, the request is cancelled and returns an error.

```
package main
import (
    "context"
    "fmt"
    "io/ioutil"
    "net/http"
    "time"
)

func main() {
    ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
    defer cancel()
    req, err := http.NewRequestWithContext(ctx, "GET", "https://api.example.com/health", nil)
    if err != nil {
        fmt.Println("Error creating request:", err)
        return
    }
    resp, err := http.DefaultClient.Do(req)
    if err != nil {
        fmt.Println("Request error:", err) // will show timeout or cancellation error
        return
    }
    defer resp.Body.Close()
    body, _ := ioutil.ReadAll(resp.Body)
    fmt.Println("Status:", resp.StatusCode)
    fmt.Println("Response:", string(body))
}
```

Example Output

Success:
Status: 200
Response: ("status":"ok")

Timeout Error:
Request error:
context deadline
exceeded

Key Points:

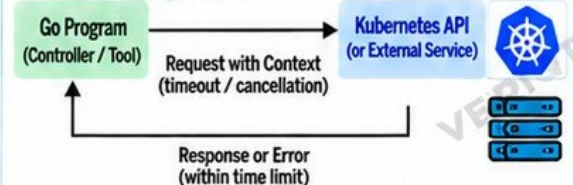
- The request will stop after 5 seconds if no response is received.
- Always handle the error and check for timeout errors.
- Use contexts with HTTP, database and other external calls.

6. Avoiding Indefinitely Hanging Operations

- Always use timeouts or deadlines for network calls, file operations and other external operations.
- Do not rely on default settings that can wait forever.
- Handle context errors (e.g. `context.DeadlineExceeded`).

7. Connection to Kubernetes Controllers and APIs

- Kubernetes controllers, operators and clients use contexts to control the lifetime of API requests.
- Contexts help with retries, cancellation, and graceful shutdowns.
- Use timeouts when calling the Kubernetes API to avoid hanging reconciliation loops.
- The same pattern applies to cloud APIs, databases, and other infrastructure services.



8. Production Rule

- External operations should generally have bounded execution.
- Always set reasonable timeouts.
- Handle cancellation and errors properly.
- Do not allow your automation to hang indefinitely.

9. Junior Engineer Tip

- Always use a context with a timeout for external calls.
- Check for `context.DeadlineExceeded` errors in your error handling.
- Use `defer cancel()` to avoid leaks.
- Keep timeouts reasonable (not too short and not too long).

10. Key Takeaways

- ✓ Use `context.Context` for deadlines, timeouts and cancellation.
- ✓ Set timeouts for HTTP, database and external API calls.
- ✓ Always handle context errors.
- ✓ Cancellation helps stop work when it is no longer needed.
- ✓ Bounded execution makes automation more reliable and safe.
- ✓ This pattern is widely used in Kubernetes and production systems.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Testing DevOps Automation

18 VERIQTA

1. Why Infrastructure Tooling Needs Tests

- Infrastructure automation can change critical systems (servers, networks, databases, cloud resources).
- Bugs can cause outages, data loss or high costs.
- Tests help catch mistakes early, before running in production.
- Tests make your automation more reliable, maintainable and safer to change.
- They give confidence when refactoring or adding new features.

3. Test Functions

- Test functions start with Test and take *testing.T.
- Use `t.Errorf()` or `t.Fatalf()` to report failures.
- Keep tests focused and clear.

```
package main
import "testing"

func TestAdd(t *testing.T) {
    result := 2 + 2
    if result != 4 {
        t.Errorf("Expected 4, got %d", result)
    }
}
```

Key Points:

- Function name must start with Test.
- Use *testing.T to report results.
- `t.Errorf()` continues the test.
- `t.Fatalf()` stops the test immediately.

5. Testing Successful and Failed Inputs

- Test both valid (successful) and invalid (failed) inputs.
- Ensure your function handles errors correctly.
- Check both the returned result and the error.

```
func TestValidateConfig(t *testing.T) {
    tests := []struct {
        name      string
        input      string
        wantErr    bool
    }{
        {"valid config", "api.example.com", false},
        {"empty input", "", true},
        {"invalid format", "http://bad-url", true},
    }
    for _, tt := range tests {
        t.Run(tt.name, func(t *testing.T) {
            err := ValidateConfig(tt.input)
            if (err != nil) != tt.wantErr {
                t.Errorf("unexpected error result: %v", err)
            }
        })
    }
}
```

Test Cases:

- ✓ Valid input should succeed.
- ✗ Empty input should fail.
- ✗ Invalid input should fail.

2. go test and _test.go

- Go has a built-in testing framework.
- Test files must end with `_test.go`.
- Run tests using `go test`.
- No additional libraries are required for basic testing.

```
# Run all tests in the current directory
go test

# Run tests with verbose output
go test -v

# Run a specific test
go test -run TestValidateConfig
```

Key Points:

- Test files end with `_test.go`.
- Test functions start with Test.
- Use `go test` to run tests.
- Use `-v` for detailed output.

4. Table-Driven Testing Introduction

- Table-driven tests allow you to test multiple inputs and expected outputs in one test function.
- This keeps tests concise and easy to maintain.

```
package main
import "testing"

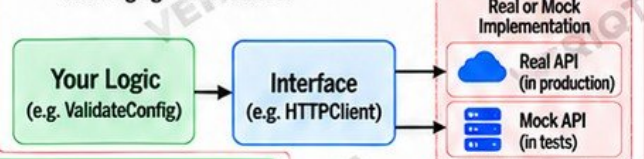
func TestAddTable(t *testing.T) {
    tests := []struct {
        name string
        a int
        b int
        expect int
    }{
        {"positive numbers", 2, 3, 5},
        {"zero values", 0, 0, 0},
        {"negative numbers", -1, -1, -2},
    }
    for _, tt := range tests {
        t.Run(tt.name, func(t *testing.T) {
            result := tt.a + tt.b
            if result != tt.expect {
                t.Errorf("expected %d, got %d", tt.expect, result)
            }
        })
    }
}
```

Benefits:

- Test multiple scenarios.
- Easy to add new test cases.
- Keeps tests organized.
- Common pattern in Go projects.

6. Separating Logic from External Dependencies

- Keep your logic separate from external systems (APIs, file systems, databases) to make it easier to test.
- Use interfaces to mock external dependencies.
- This allows you to test your logic without making real network calls or changing real resources.



7. Practical Example: Test a Configuration-Validation Function

- This example validates a simple configuration value (API endpoint).
- It checks for empty values and invalid formats.

```
config.go
package main
import (
    "strings"
)

func ValidateConfig(endpoint string) error {
    if endpoint == "" {
        return fmt.Errorf("endpoint cannot be empty")
    }
    if !strings.HasPrefix(endpoint, "http://") &&
        !strings.HasPrefix(endpoint, "https://") {
        return fmt.Errorf("endpoint must start with http:// or https://")
    }
    return nil
}
```

```
config_test.go
package main
import "testing"

func TestValidateConfig(t *testing.T) {
    tests := []struct {
        name      string
        input      string
        wantErr    bool
    }{
        {"valid https", "https://api.example.com", false},
        {"valid http", "http://localhost:8080", false},
        {"empty", "", true},
        {"invalid format", "api.example.com", true},
    }
    for _, tt := range tests {
        t.Run(tt.name, func(t *testing.T) {
            err := ValidateConfig(tt.input)
            if (err != nil) != tt.wantErr {
                t.Errorf("unexpected error: %v", err)
            }
        })
    }
}
```

Run the Tests:

go test -v

```
=== RUN TestValidateConfig
--- PASS: TestValidateConfig (0.00s)
--- PASS: TestValidateConfig/valid_https
--- PASS: TestValidateConfig/valid_http
--- PASS: TestValidateConfig/empty
--- PASS: TestValidateConfig/invalid_format
PASS
ok   example.com/config   0.001s
```

8. CI/CD Connection

- Tests should run automatically in your CI/CD pipeline (GitHub Actions, GitLab CI, Jenkins, etc.).
- If tests fail, the pipeline should stop deployments.
- This prevents broken automation from reaching production.



9. Production Rule

- External operations should generally have bounded execution.
- Always use timeouts, input validation and tests.
- Do not deploy untested automation to production.

10. Junior Engineer Tip

- Write tests for your core logic first.
- Use table-driven tests.
- Mock external dependencies.
- Aim for both success and failure test cases.
- Run tests frequently while developing.

11. Key Takeaways

- Tests make your infrastructure automation safer and reliable.
- Use `go test` and `_test.go`.
- Write clear test functions and use table-driven tests.
- Separate logic from external dependencies.
- Tests should be part of your CI/CD pipeline.
- Do not skip tests, especially for infrastructure tools.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



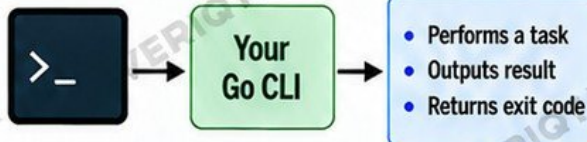
@veriqta

Build a Production-Friendly Go CLI

19 VERIQTA

1. What a CLI Is

- A CLI (Command Line Interface) is a program you run from the terminal.
- It accepts arguments, performs a task, and returns output.
- Useful for infrastructure automation, DevOps tools, and system administration.
- CLIs are lightweight, fast, and easy to integrate into scripts and CI/CD pipelines.



2. Command-Line Arguments and Flags

- Use the `flag` package to define and parse flags.
- Flags make your CLI flexible and user-friendly.
- You can also accept positional arguments (non-flag inputs).

```
package main
import (
    "flag"
    "fmt"
)

func main() {
    name := flag.String("name", "world",
        "Name to greet")
    verbose := flag.Bool("v", false,
        "Enable verbose output")
    flag.Parse()

    if *verbose {
        fmt.Println("[INFO] Saying hello...")
    }
    fmt.Printf("Hello, %s!\n", *name)
}
```

Example Usage:

```
go run main.go
--name DevOps

go run main.go -v

go run main.go
--help
```

Key Points:

- Use `flag.String()`, `flag.Bool()`, etc.
- Call `flag.Parse()`
- Use `--help` to show usage information.

3. Helpful Usage Output

- Provide clear usage instructions.
- Users should understand how to use your CLI.
- The `flag` package automatically generates basic help output when you run `--help`.

```
$ go run main.go --help

Usage of /tmp/go-build/main:
-name string
    Name to greet (default "world")
-v
    Enable verbose output (default false)
```

4. Exit Codes

- Exit codes indicate success or failure.
- Use `os.Exit()` to set the code.
- Common codes:
 - 0 = success
 - non-zero = error

Common Exit Codes

Code	Meaning
0	Success
1	General error
2	Invalid arguments
3	Configuration error

5. Logging

- Use structured and informative logs.
- Include timestamps and log levels.
- Use the log package or a structured logging library for production (e.g. zap, logrus).

```
package main
import (
    "log"
)

func main() {
    log.Println("Starting CLI...")
    log.Printf("Processing %s", "task")
    log.Println("Task completed")
}
```

6. Configuration

- Allow configuration through flags, config files, or environment variables.
- Common formats: JSON, YAML, or simple key-value files.
- Configuration makes your CLI flexible across environments.

Example: Read a Config File (JSON)

```
type Config struct {
    APIEndpoint string `json:"api_endpoint"`
    Timeout      int    `json:"timeout"`
}

// Load configuration from a file
// (error handling omitted for brevity)
```

7. Error Messages

- Provide clear and actionable error messages.
- Explain what went wrong and how to fix it.
- Avoid confusing or vague errors.

```
$ go run main.go --name ""
Error: name cannot be empty

$ go run main.go --config missing.json
Error: cannot read config file:
open missing.json: no such file or directory
```

Good Error Message Guidelines:

- Be specific and clear.
- Include the failing resource (file, URL, etc.).
- Suggest how to fix the problem.

8. Cross-Compilation Introduction

- Go can build binaries for different operating systems and architectures.
- Useful for distributing your CLI to Linux, Windows, macOS, and different CPU types.
- Use `GOOS` and `GOARCH` environment variables.

Build for Different Platforms

```
# Linux (amd64)
GOOS=linux GOARCH=amd64 go build -o mycli

# Windows (amd64)
GOOS=windows GOARCH=amd64 go build -o mycli.exe

# macOS (arm64)
GOOS=darwin GOARCH=arm64 go build -o mycli
```

9. Build a Portable Binary

- Use `go build` to create a binary.
- Use `-o` to specify the output name.
- Strip debug information for smaller size (optional).

```
# Build for current platform
go build -o mycli

# Build a smaller binary (optional)
go build -ldflags="-s -w" -o mycli
```

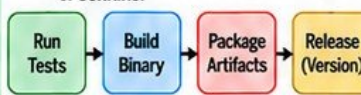
Result:

A single executable file (mycli) that you can run on the target platform without needing Go installed.



10. CI/CD Connection

- Your CLI should be tested, built, packaged, and released using CI/CD.
- This ensures consistent, repeatable releases.
- You can use GitHub Actions, GitLab CI, or Jenkins.



11. Key Takeaways

- ✓ A CLI is a powerful tool for DevOps automation.
- ✓ Use flags for flexible input.
- ✓ Provide helpful usage output.
- ✓ Use clear exit codes and error messages.
- ✓ Support configuration files or environment variables.
- ✓ Learn cross-compilation with `GOOS` and `GOARCH`.
- ✓ Build portable binaries for multiple platforms.
- ✓ Integrate testing and builds into your CI/CD pipeline.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

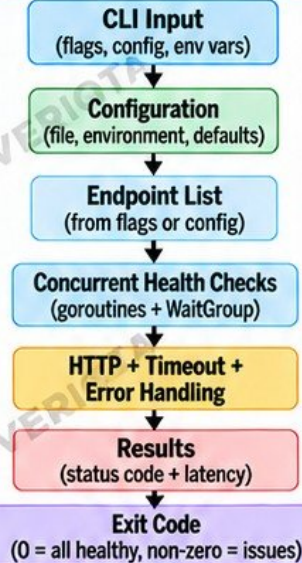
Capstone: DevOps Service Health CLI

20 VERIQTA

1. Project Overview

- Bring the notebook together by building a small real tool.
- The CLI accepts several service endpoints and:
 - Loads configuration
 - Reads environment variables
 - Makes HTTP requests
 - Applies timeouts
 - Checks endpoints concurrently
 - Records status codes and latency
 - Handles unreachable services
 - Produces readable results
 - Returns an appropriate exit status
 - Can run locally or inside CI/CD

2. Architecture



3. CLI Usage and Flags

- Use flags to provide endpoints, config file, timeout, and output format.
- Support both command-line arguments and a config file.
- Provide helpful usage information with --help.

```
$ go run main.go --help
Usage of ./healthcheck:
  -config string      Path to config file (default "config.yaml")
  -endpoints string   Comma-separated endpoints (e.g. https://api.example.com)
  -timeout duration   Request timeout (default 5s)
  -format string      Output format: table|json (default "table")
  -v                 Enable verbose output (default false)
```

4. Configuration and Environment Variables

- Load endpoints from a config file or command flags.
- Use environment variables for sensitive values and defaults.
- Common formats: JSON or YAML.
- Environment variables example:

```
HEALTH_TIMEOUT=5s
DEFAULT_ENDPOINTS=https://api.example.com
API_TOKEN=your-token-here
```

Example Config File (YAML)

```
endpoints:
  - name: api
    url: https://api.example.com/health
  - name: web
    url: https://web.example.com/health
timeout: 5s
format: table
```

5. Concurrent Health Checks

- Use goroutines to check multiple endpoints at the same time.
- Use `sync.WaitGroup` to wait for all checks to finish.
- Apply timeouts and proper error handling.

```
func checkEndpoint(url string, timeout
time.Duration) wg *sync.WaitGroup, resultCh
chan<- Result {
    defer wg.Done()
    ctx, cancel := context.WithTimeout(
        context.Background(), timeout)
    defer cancel()
    start := time.Now()
    resp, err := http.Get(url)
    latency := time.Since(start)
    if err != nil {
        resultCh <- Result{URL: url, Status: "error",
            Error: err.Error(), Latency: latency}
        return
    }
    defer resp.Body.Close()
    resultCh <- Result{URL: url, Status:
        fmt.Sprintf("%d", resp.StatusCode),
        Latency: latency}
}
```

6. Results Output

- Show a clear and readable summary.
- Include endpoint, status code, latency and status.
- Support table output (default) and JSON output.

ENDPOINT	STATUS	CODE	LATENCY
https://api.example.com	UP	200	120ms
https://web.example.com	UP	200	95ms
https://db.example.com	DOWN	error	-
https://cache.example.com	UP	200	80ms

JSON Output Example

```
{
  "results": [
    {
      "url": "https://api.example.com",
      "status": "up",
      "code": 200,
      "latency": "120ms"
    },
    {
      "url": "https://db.example.com",
      "status": "down",
      "error": "timeout",
      "latency": "5s"
    }
  ],
  "summary": {
    "total": 2,
    "up": 1,
    "down": 1
  }
}
```

7. Exit Codes

- Return exit code 0 when all endpoints are healthy.
- Return a non-zero exit code when any endpoint fails.

Condition	Exit Code
All services healthy	0
One or more services down	1
Configuration error	2
Invalid arguments	3

8. Handle Unreachable Services

- Use timeouts to avoid hanging.
- Handle network errors, DNS errors, and HTTP errors.
- Continue checking other endpoints even if one fails.

```
https://db.example.com -> ERROR
error: context deadline exceeded
https://cache.example.com -> ERROR
error: dial tcp: no route to host
```

9. Real Engineering Workflows

- Use this CLI for deployment verification.
- Run it as a smoke test after a deployment.
- Integrate it into CI/CD pipelines as a quality gate.
- Use it as a monitoring helper or operational tool.
- Easily extend it to check more endpoints or services.



10. Build and Run

- Build a binary for your platform.
- Cross-compile for different operating systems.

```
# Build for current platform
go build -o healthcheck main.go
# Cross-compile for Linux (amd64)
GOOS=linux GOARCH=amd64 go build -o healthcheck main.go
# Cross-compile for Windows (amd64)
GOOS=windows GOARCH=amd64 go build -o healthcheck.exe main.go
```

11. CI/CD Connection

- Add tests for your functions.
- Build the binary.
- Run the CLI against key services.
- Fail the pipeline if endpoints are unhealthy.
- Package and release the binary.



12. Key Takeaways

- ✓ A CLI can automate real operational tasks.
- ✓ Use flags, configuration and environment variables.
- ✓ Apply timeouts and handle errors properly.
- ✓ Use concurrency for faster checks.
- ✓ Provide clear output and correct exit codes.
- ✓ This tool is useful for deployments, smoke tests, monitoring helpers and CI/CD gates.
- ✓ It brings together all the concepts from this notebook.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta